Lab 4 Report: Gridworld Value Iteration and Policy Iteration

1. Introduction

This lab implements and compares two classic dynamic programming methods for Markov Decision Processes (MDPs) on a 5x5 Gridworld: Value Iteration (VI) and Policy Iteration (PI). The goals are to compute the optimal value function, derive the optimal policy, and confirm that PI converges to the same policy as VI from multiple random starts (bonus check).

2. Problem Setup

- **Environment**: 5x5 Gridworld with stochastic transitions.
- **Actions**: Up, Down, Left, Right.
- **Rewards**: Matrix defined by two variable rewards at the top corners and fixed values elsewhere.
- **Terminal states**: Top-left and top-right cells.
- **Transition model**: Intended action succeeds with probability 0.7; the other three actions each occur with probability 0.1.
- **Discount factor**: $\gamma = 0.95$ (default).

3. MDP Formulation

Let S be the set of states and A the set of actions. The stochastic transition model is:

$$ P(s' \mid s, a) = \begin{cases} 0.7 & \text{if } s' \text{ results from intended action } a \\ 0.1 & \text{if } s' \text{ results from any other action} \end{cases} $$

The optimal value function is defined by the Bellman optimality equation:

$$ V^*(s) = \max_a \sum_{s'} P(s' \mid s, a) \left[R(s, a, s') + \gamma V^*(s') \right]. $$

4. Value Iteration (VI)

Value Iteration repeatedly applies the Bellman optimality backup until the values converge.

4.1 Algorithm Outline

1. Initialize $V(s) = 0$ for all states.
2. For each state, compute the expected return for each action (one-step lookahead).
3. Update $V(s)$ to the maximum action value.
4. Stop when the largest update is below a small threshold θ.
5. Derive a greedy policy from the final V.

4.2 Notes

- VI directly estimates the optimal value function V^*.
- The greedy policy is extracted after convergence by choosing the action with the highest expected return.

5. Policy Iteration (PI)

Policy Iteration alternates between evaluating a fixed policy and improving it greedily.

5.1 Algorithm Outline

1. Start with a random policy (or a given initial policy).
2. **Policy Evaluation**: compute V^π for the current policy.
3. **Policy Improvement**: update the policy to be greedy with respect to V^π.
4. Repeat until the policy stops changing.

5.2 Notes

- PI often converges in fewer iterations than VI because it updates the policy directly.
- Evaluation uses iterative sweeps until the value changes are below θ.

6. Bonus Check (Random Starts)

The bonus requirement checks whether Policy Iteration converges to the same policy as Value Iteration across multiple random initial policies. The implementation runs PI 10 times with different random seeds and compares each result to the VI policy. The expected outcome is that all PI runs match the VI policy.

7. GUI Extension

A Tkinter GUI was added to make the lab interactive. The GUI allows the user to:

- Enter $R1$, $R2$, and γ.
- Run Value Iteration or Policy Iteration (or both).
- View the value table and policy arrows in a text grid.
- See the bonus check result (PI random starts vs VI) inside the PI panel.

8. How to Run

1. Install requirements:

```
pip install -r rl_lab4/requirements.txt
```

2. Run the console driver:

```
python3 rl_lab4/main.py
```

3. Run the GUI:

```
python3 rl_lab4/gui.py
```

9. Discussion

Both VI and PI solve the same MDP and are guaranteed to converge to the optimal policy under standard assumptions. VI performs pure value updates and can require more sweeps; PI often converges faster because it improves the policy directly. The bonus check provides evidence that the optimal policy is unique for this task and that PI consistently finds it from different random starts.

10. Conclusion

This lab demonstrates the practical implementation of dynamic programming methods for MDPs. Value Iteration provides a direct path to V^*, while Policy Iteration alternates between evaluation and improvement. The GUI helps visualize the value function and policy structure and makes it easy to explore different reward settings.